Campus Marketplace ? Presentation Speech

Delivered by Solo Presenter for 5-Member Team | Duration: 8-10 Mins

1. Team Roles & Equal Contribution (20% Each)

MEMBER	MODULE ASSIGNED	DEDICATED SOURCE FILES
Member 1	Campus Discovery & Request Board	ItemRequestsModal.tsx, ExplorePage.tsx, data.ts
Member 2	Live Chat & Price Bargaining	MakeOfferModal.tsx, MessagesPage.tsx
Member 3	Listing CRUD & Safe Meetups	MeetupSchedulerModal.tsx, MyListingsPage.tsx, ListingCard.tsx
Member 4	Auth Security & Seller Reviews	SellerReviewsModal.tsx, ProfilePage.tsx
Member 5	Data Models & Content Moderation	ReportListingModal.tsx, SavedPage.tsx, types.ts

2. Introduction & Problem Statement (~1.5 Mins)

[Stage Cue: Welcome panel and state project title clearly]

Good morning / afternoon respected lecturers and panel members. My name is [Your Name], and today I am proud to present our software engineering project, Campus Marketplace, on behalf of our 5-member team: [Member 1], [Member 2], [Member 3], [Member 4], and myself.

Every semester, university students face heavy financial hurdles purchasing textbooks, notes, and lab gear. Existing platforms like Facebook Marketplace or Ikman lack university verification, fail to filter by campus, and carry safety risks during meetups.

We engineered Campus Marketplace: a secure peer-to-peer trading app built with React Native Expo SDK 57, Firebase Cloud Firestore, and Cloud Storage. We expanded an initial prototype into an enterprise student marketplace, divided equally into 5 dedicated modules.

3. Module 1: Campus Search & Request Board (Member 1)

[Action: Display Explore Page. Filter Moratuwa / Colombo, then tap Request Board banner]

Starting with our discovery layer, engineered by Member 1. In the initial prototype, search was basic with placeholder 'North Campus' labels. Member 1 transformed this into a realistic Sri Lankan campus ecosystem:

- University Localization: Mapped 17 Sri Lankan state universities, enabling students to filter items strictly available on their campus.
- LKR Price Range Filters: Implemented real-time Minimum and Maximum price sliders/inputs.
- Student Request Board (WTB): Developed ItemRequestsModal.tsx allowing students to post wanted requests with target budgets and urgency tags when items are not listed.

4. Module 2: Live Chat & Price Bargaining (Member 2)

[Action: Open a listing, tap 'Make Offer'. Show discount chips. Navigate to Messages tab]

Once a buyer finds an item, communication and bargaining are the next essential steps, engineered by Member 2. In the initial project, Messages was an empty 20-line UI stub with zero database code.

- Price Bargaining Engine: Built MakeOfferModal.tsx with discount presets (-5%, -10%, -15%, -20%), instant LKR savings calculation, and deal notes like 'Can pick up today at library'.
- Real-Time Firestore Chat: Built MessagesPage.tsx subscribing to Firestore conversations and chronological message subcollections with server timestamps and unread previews.

5. Module 3: Seller CRUD & Safe Meetups (Member 3)

[Action: Show 'Sell' form with photo picker. Show My Listings Edit/Delete. Tap 'Safe Meetup']

Next, for inventory management and physical logistics, Member 3 led development. In the base project, users could only create listings with hardcoded dummy photos, with no way to edit or delete items.

- Listing CRUD & Cloud Storage: Connected expo-image-picker and Firebase Cloud Storage for authentic mobile gallery photo uploads, plus Edit listing forms and Delete with native alerts.
- Safe Campus Meetup Coordinator: Built MeetupSchedulerModal.tsx designating verified safe zones inside campuses (Library Lobby, Central Canteen, Security Gate) with date/time scheduling and safety guidelines.

6. Module 4: Auth Security & Seller Reviews (Member 4)

[Action: Show Profile Sign In / Sign Up toggle. Tap 'Reviews' on a listing. Tap Back button]

To guarantee student trust and eliminate scammers, Member 4 engineered our security and reputation layer. The initial base template supported only anonymous guest logins with zero seller accountability.

- Email & Password Auth: Implemented full authentication with form validation, session persistence, and Firestore profile synchronization.
- Seller Reviews System: Created SellerReviewsModal.tsx featuring 5-star ratings, category satisfaction metrics, student trust badges (Verified Student, Fast Responder), and peer testimonials.
- Modal Touch Freeze Fix: Resolved a critical Android Expo Go bug where stacked native modals locked touch focus, architecting smooth single-modal view navigation with Back buttons.

7. Module 5: Data Models & Scam Shield (Member 5)

[Action: Tap heart to toggle Saved item. Tap 'Report suspicious listing' under details]

Finally, application stability, data integrity, and scam moderation were spearheaded by Member 5:

- TypeScript Data Architecture: Architected types.ts declaring strict shared models for Listing, Conversation, Message, ItemRequest, Offer, MeetupPlan, SellerReview, and ListingReport.
- Shortlist Synchronization: Optimized real-time Firestore saved-items subcollection sync with instant optimistic UI.
- Content Moderation Shield: Built ReportListingModal.tsx allowing confidential reporting of fraudulent items, stolen property, and exam leaks for administrative review.

8. Architecture Highlights & Conclusion

In summary: each member's feature is modularly isolated under src/features/ with zero merge conflicts. All Firebase security rules protect user data. Our TypeScript codebase compiles cleanly with zero errors (npx tsc --noEmit).

Our 5-member team successfully delivered a production-ready, secure campus marketplace for Sri Lankan university students. Thank you very much, and I am now ready for your questions!

9. Viva Panel Quick Answers Cheatsheet

- Q on Member 1 (Search): Uses in-memory & Firestore queries matching listing.university against the active campus state.
- Q on Member 2 (Chat): Root 'conversations' collection with memberIds array, nested 'messages' subcollection ordered by server timestamp.
- Q on Member 3 (Storage): Images are binary blobs uploaded to Firebase Cloud Storage; only the download URL string is stored in Firestore.
- Q on Member 4 (Modal Fix): Native Android Dialogs cannot stack without trapping focus. Solved via internal view routing in a single modal.
- Q on Member 5 (TypeScript): Strict types in types.ts guaranteed 5 developers worked concurrently with zero runtime data mismatch errors.